isec
Politécnico de Coimbra

Licenciatura em Engenharia Informática – 2022/23

Programação

5: Ficheiros

Francisco Pereira (xico@isec.pt)

Ficheiros

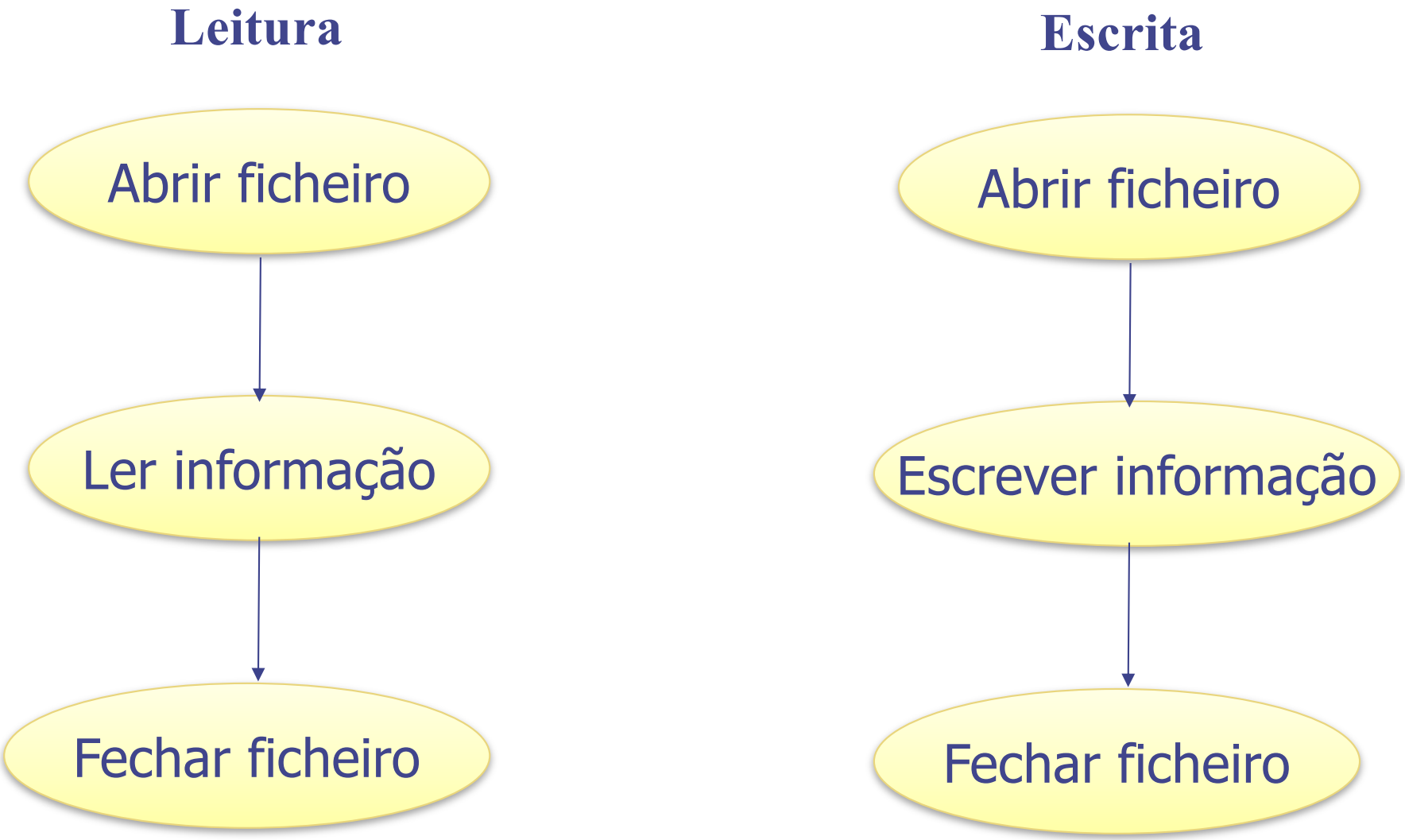
isec
Politécnico de Coimbra

- Vantagens
  - Armazenamento permanente
  - Leitura / Escrita de qualquer tipo de dados
  - Tamanho variável
  - Acesso sequencial / Acesso direto
- Limitações
  - Acesso lento

Tipos de Ficheiros

- Que tipo de ficheiro utilizar?
  - Ficheiro binário:
    - Conjuntos de bytes que representam qualquer tipo de dados: caracteres, inteiros, estruturas, reais, ...
    - Não existe a noção de linha.
  - Ficheiro de texto:
    - Conjunto de caracteres divididos por linhas.
    - Editáveis.

Operações básicas

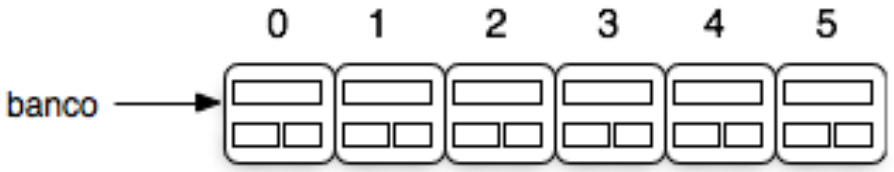


`<stdio.h>` possui funções para cada uma destas tarefas.

Ficheiros Binários

Exemplo 1

Armazenar num ficheiro binário os clientes guardados no array de estruturas banco



Array Dinâmico

```
typedef struct dados
        cliente;

struct dados {
   char nome[100];
   char nconta[15];
   int montante;
};

int main(){
   cliente *banco = NULL;
   int total=0;

   ...


   guardaDados(banco, total);
   free(banco);
   return 0;
}
```

Exemplo 1

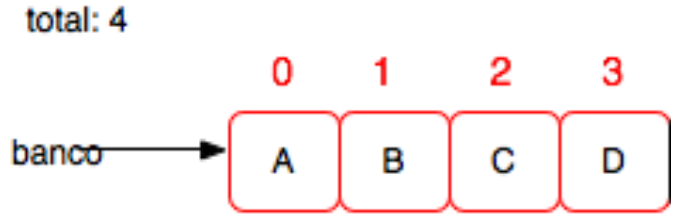
- Usar um ficheiro binário chamado `banco.dat`
  - Argumentos: ponteiro para o início do vetor e número de clientes.

```
void guardaDados(cliente *b, int total);
```

- Estratégia:
  - Abrir ficheiro `banco.dat` para escrita.
  - Guardar as estruturas no ficheiro.
  - Fechar o ficheiro.

Exemplo 1

- Duas alternativas para armazenar a informação:



banco.dat

4

A

B

C

D

Esta é a melhor opção!

ou

banco.dat

A

B

C

D

Abrir um ficheiro

Uma variável do tipo `FILE*` recebe o resultado devolvido, ficando associada ao ficheiro aberto.

Nome físico

```
FILE* fopen(char* nome, char* modo);
```

Devolve NULL em caso de erro

Modo de acesso	
Acesso	**Tipo de ficheiro**
r: leitura	**b**: binário
w: escrita	**t**: texto (por defeito)
a: acrescentar	

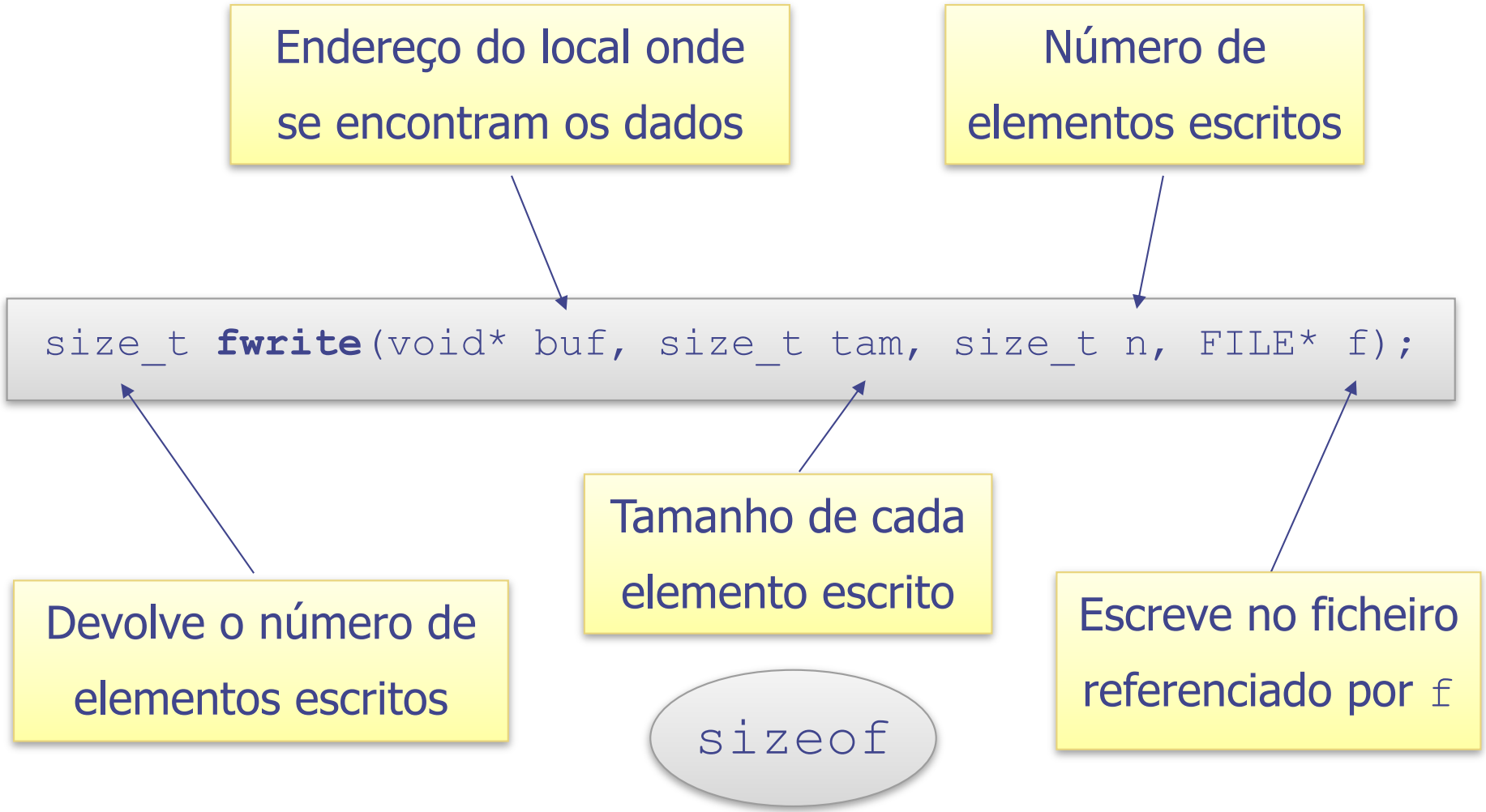
Fechar um ficheiro

Variável do tipo `FILE*` associada ao ficheiro

```
int fclose(FILE* f);
```

Devolve 0 se a operação correr bem

Escrever informação



Função `guardaDados()`: versão 1

```
void guardaDados(cliente *b, int total){
   FILE *f;
   int i;

   f = fopen("banco.dat", "wb");
   if(f==NULL)
   {
       printf("Erro no acesso ao ficheiro\n");
       return;
   }
   fwrite(&total, sizeof(int), 1, f);
   for(i=0; i<total; i++)
       fwrite((b+i), sizeof(cliente), 1, f);
   fclose(f);
}
```

Guarda o número de estruturas no início do ficheiro

Função `guardaDados()`: versão 2

```
void guardaDados(cliente *b, int total){
   FILE *f;
   int i;

   f = fopen("banco.dat", "wb");
   if(f==NULL)
   {
       printf("Erro no acesso ao ficheiro\n");
       return;
   }
   fwrite(&total, sizeof(int), 1, f);
   fwrite(b, sizeof(cliente), total, f);
   fclose(f);
}
```

Escreve todas as estruturas com uma única instrução

Função `leDados()`: versão 1

Função que recupera a informação armazenada no ficheiro

```
cliente* leDados(int *total);
```

- Recebe endereço da variável inteira onde deve colocar o número de clientes
- Devolve endereço do array dinâmico (NULL em caso de erro)
- É a função que efetua a alocação do array dinâmico

```
int main(){
   cliente *banco = NULL;
   int total=0;

   banco = leDados(&total);

   ...

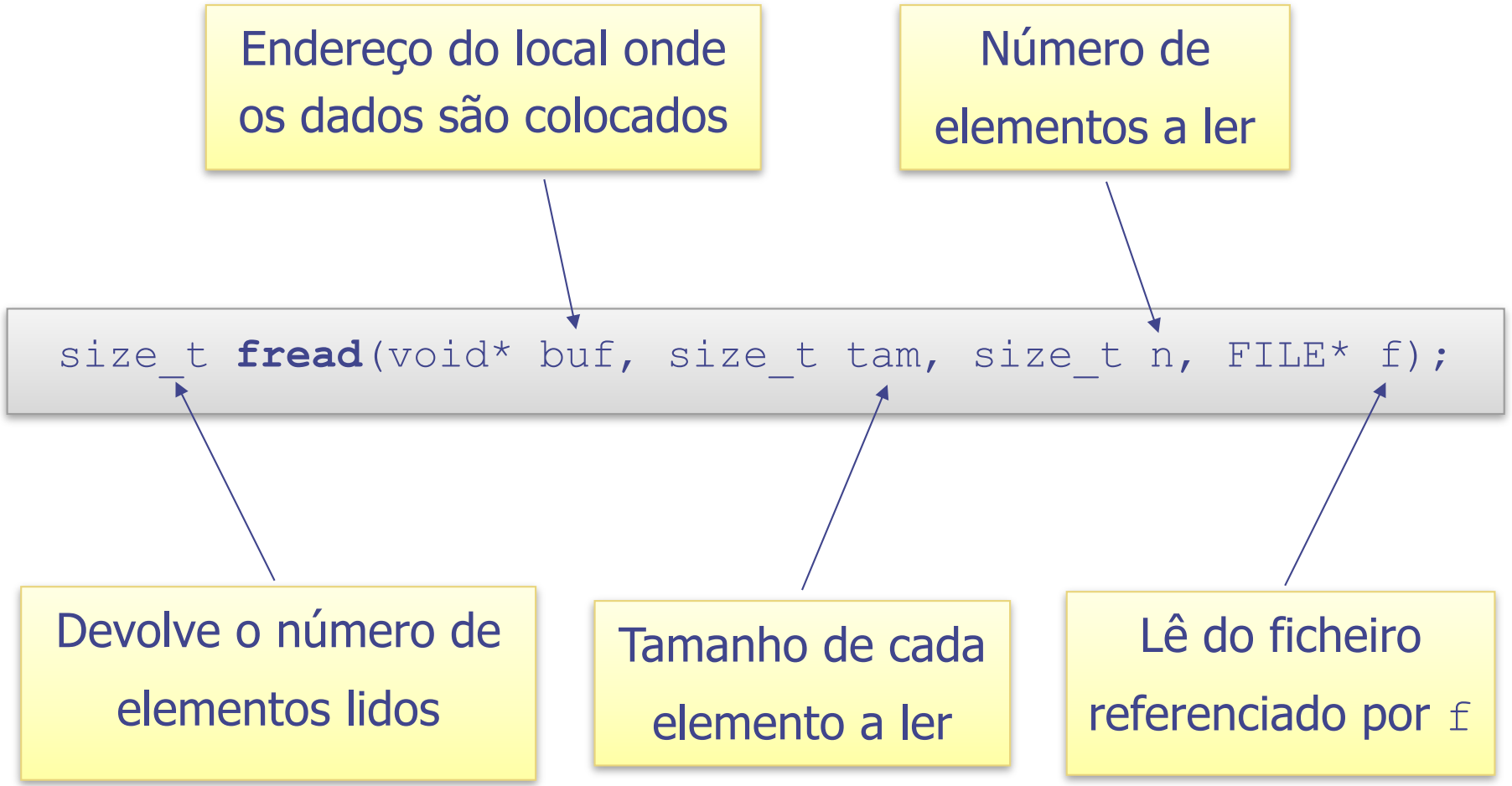
}
```

Função `leDados()`: versão 1

isec
Politécnico de Coimbra

- Estratégia:
  1. Abrir ficheiro `banco.dat` para leitura.
  2. Ler o número de estruturas armazenadas
  3. Alocar vetor dinâmico
  4. Ler as estruturas do ficheiro para o vetor
  5. Fechar o ficheiro.
  6. Devolver vetor dinâmico

Ler informação



Função `leDados()`: versão 1

```
cliente* leDados(int *total){
   FILE *f;
    cliente *b = NULL;

    *total = 0;
   f = fopen("banco.dat", "rb");
   if(f==NULL){
       printf("Erro no acesso ao ficheiro\n");
       return NULL;
   }
   fread(total, sizeof(int), 1, f);
    b = malloc(sizeof(cliente) * (*total));
    if(b == NULL){
       fclose(f); *total = 0;
        return NULL;
    }
   fread(b, sizeof(cliente), *total, f);
   fclose(f);
    return b;
}
```

Tem que fazer a alocação inicial do array

Exemplo 1

- E se o ficheiro não tiver o valor inteiro no início?

Não sabemos quantas estruturas existem no ficheiro.

Função `guardaDados()`: versão 3

```
void guardaDados(cliente *b, int total){
   FILE *f;
   int i;

   f = fopen("banco.dat", "wb");
   if(f==NULL)
   {
       printf("Erro no acesso ao ficheiro\n");
       return;
   }
   fwrite(b, sizeof(cliente), total, f);
   fclose(f);
}
```

Não é guardado o valor inteiro

Função `leDados()`: versão 2

isec
Politécnico de Coimbra

- Neste caso não se sabe o número de estruturas
- É impossível fazer a alocação inicial
- Duas opções:
  - Ler uma estrutura de cada vez e ir realocando
  - Criar uma função auxiliar para contar o número de estruturas e depois proceder como na função original

Função auxiliar

```
int getTotal(char *nomeF){
   FILE *f;
   cliente aux;
   int total = 0;

   f = fopen(nomeF, "rb");
   if(f==NULL)
        return 0;

   while(fread(&aux, sizeof(cliente), 1, f) == 1)
        total++;

   fclose(f);
   return total;
}
```

É lida uma estrutura de cada vez

Devolve o número de estruturas armazenadas no ficheiro

Função `leDados`: versão 2

```
cliente* leDados(int *total){
   FILE *f;
    cliente *b = NULL;

    *total = getTotal("banco.dat");
   if (*total == 0)  return NULL;
   f = fopen("banco.dat", "rb");
   if(f==NULL){
       printf("Erro no acesso ao ficheiro\n"); return NULL;
   }
   b = malloc(sizeof(cliente) * (*total));
    if(b == NULL){
        fclose(f); *total = 0;
        return NULL;
    }
   fread(b, sizeof(cliente), *total, f);
   fclose(f);
    return b;
}
```

Detetar Final de Ficheiro

- A função anterior deteta o final de ficheiro após falha da função de leitura

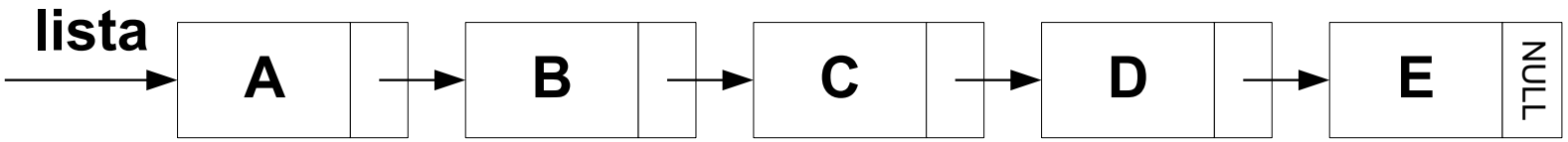
- Outra possibilidade
  - Função que verifica se foi efetuada uma leitura para além do final do ficheiro

```
int feof(FILE* f);
```

Devolve valor ≠ 0 se o indicador de final de ficheiro foi atingido

Exemplo 2

Armazenar num ficheiro binário os livros de uma lista ligada simples

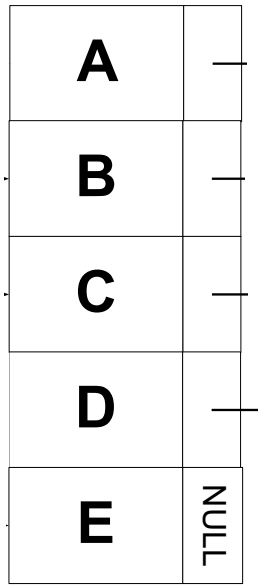



```
int main(){
   plivro lista = NULL;

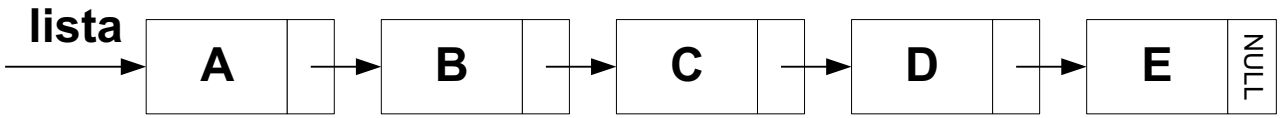
   ...

   guardaLista(lista, "lista.bin");
   libertaLista(lista);
   return 0;
}
```

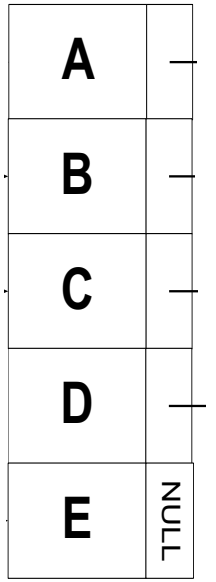
lista.bin



Exemplo 2



lista.bin



Outras possibilidades a considerar:

1. Guardar o número de estruturas no início
2. Não guardar o campo *prox* das estruturas

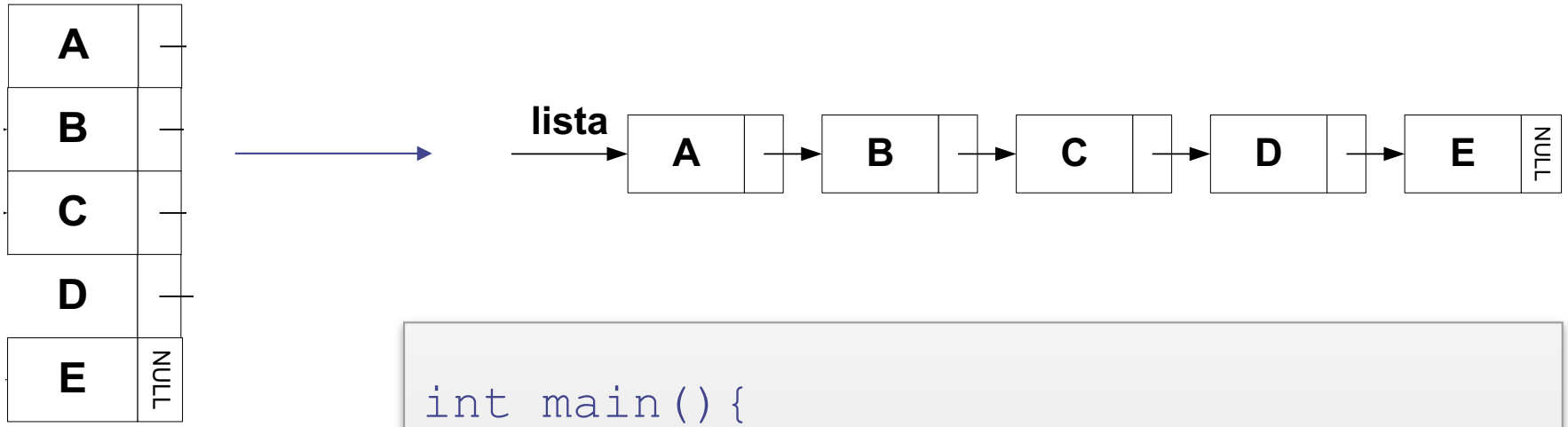
Função `guardaLista()`

```
int guardaLista(plivro p, char *nomeF){
    FILE *f;

    f = fopen(nomeF, "wb");
    if(f==NULL)
        return -1;
    while(p != NULL){
        fwrite(p, sizeof(livro), 1, f);
        p = p->prox;
    }
    fclose(f);
    return 1;
}
```

Exemplo 2

Recuperar a lista ligada a partir do ficheiro




```
int main(){
   plivro lista = NULL;

   lista = recuperaLista("lista.bin");
   mostra_info(lista);
   ...

}
```

Função `recuperaLista()`

```
plivro recuperaLista(char *nomeF);
```

- Esta função vai criar uma lista ligada a partir da informação que está armazenada no ficheiro
- Estratégia:
    1. Criar lista ligada
    2. Abrir ficheiro para leitura.
    3. Repetir
        1. Ler próxima estrutura do ficheiro
        2. Alocar, preencher e inserir no final da lista
    4. Fechar o ficheiro.
    5. Devolver lista

Função `recuperaLista()`

```
plivro recuperaLista(char* nomeF){
    plivro novo, aux, p = NULL;
    livro liv;
    FILE *f;

    f = fopen(nomeF, "rb");
    if(f == NULL) return NULL;

    while(fread(&liv, sizeof(livro), 1, f) == 1){
        novo = malloc(sizeof(livro));
        if(novo == NULL){
            fclose(f); return p;
        }
        *novo = liv;
        novo->prox = NULL;
        p = insereNoFinal(p, novo);
    }
    fclose(f);
    return p;
}
```

Ficheiros de Texto

Ficheiros de texto

- Ficheiros de texto mais comuns num programa em C:
  - `stdin`
  - `stdout`
  - `stderr`
- O que pode ser feito?
  - Leitura/escrita de caracteres
  - Leitura/escrita de conjuntos de caracteres (palavras/linhas)
  - Leitura/escrita formatada

Ficheiros de texto

- Escrita do caracter `c` no ficheiro `f`:

```
int fputc(int c, FILE* f);
```

Devolve caracter escrito (EOF se erro)

- Leitura do próximo caracter do ficheiro `f`:

```
int fgetc(FILE* f);
```

Devolve caracter lido (EOF se erro ou final do ficheiro)

Ficheiros de texto

- Escrita de uma linha num ficheiro:

```
int fputs(char* s, FILE* f);
```



- Leitura de um conjunto de caracteres de um ficheiro:

```
char* fgets(char* s, int n, FILE* f);
```



Termina quando surgir '\n' ou atingir n-1 caracteres

Funções para escrita/leitura formatada

- Escrita de dados num ficheiro:

```
int fprintf(FILE* f, ...);
```

Restantes argumentos são idênticos aos da função `printf()`

- Leitura de dados de um ficheiro:

```
int fscanf(FILE* f, ...);
```

Restantes argumentos são idênticos aos da função `scanf()`

Exemplo 1

```
#include <stdio.h>

int main()
{
   FILE *f;
   int i = 10;
   float x = -234.5;
   char st[20] = "Ola Mundo";

   f = fopen("ex1.txt", "w");
   if(f != NULL)
   {
       fprintf(f, "%s\n%d\t%3.3f\n", st, i, x);
       fputs("Ola outra vez", f);
       fputc('\n', f);
       fputc('A', f);
       fclose(f);
   }
   return 0;
}
```

Exemplo 2

```
#include <stdio.h>

int main()
{
 FILE *f;
 char st1[20], st2[20];
 int i;
 float x;

 f = fopen("e1.txt", "r");
 if(f != NULL)
 {
   fscanf(f, " %d ", &i);
   fgets(st1, 20, f);
   fscanf(f, " %f %c", &x, st2);
   st2[1] = fgetc(f);
   fgets(st2+2, 5, f);
   fclose(f);
 }
 return 0;
}
```

e1.txt

```
34
Ola Mundo
-12.467abcdefghij
```

Exemplo 3

- Processar um ficheiro de texto contendo as apostas efetuadas num concurso do euromilhões
- Informação de um apostador está dividida em 2 linhas:
  - Nome
  - 7 valores inteiros

dados.txt

```
Ana Mendes
3  5  7  12  20  1  3

Carlos Antunes Lima
2  5  12  21  24  4  5

Luis Silva
5  9  10  17  25  2  4
```

Exemplo 3

- Leitura da informação de um apostador:

```
int i, a[7];
char st[100];
FILE *f;

...

fscanf(f, " %[^\n]", st);

for(i=0; i<7; i++)
   fscanf(f, "%d", &a[i]);

...
```

Exemplo 4

- Inicializar algumas posições de uma matriz quadrada (tipo *float*):
  - As posições e os valores a preencher estão armazenados num ficheiro

```
#define N 4

float m[N][N] = {0.0};
int i,j;

// Chamar função de leitura

for(i=0; i<N; i++){
   for(j=0; j<N; j++)
    printf("%.3f\t",m[i][j]);
   putchar('\n');
```

mat.txt

```
1   2   1283.434
1   0   -122.122
4   5   33.413
0   3   -319.605
```

mat.bin

```
1   2   1283.434
1   0   -122.122
4   5   33.413
0   3   -319.605
```

Leitura do ficheiro de texto

isec
Politécnico de Coimbra

```
void leTXT(char *nome, int n, float mat[n][n]){
    FILE *f;
    int x[2];
    float v;

    f = fopen(nome, "r");
    if(f==NULL){
      printf("Erro no acesso ao ficheiro\n");
      return;
    }
    while(fscanf(f, "%d %d %f", x, x+1, &v) == 3){
        if(x[0]>=0 && x[0]<n && x[1]>=0 && x[1]<n)
            mat[x[0]][x[1]] = v;
    }
    fclose(f);
}
```

Leitura do ficheiro binário

```
void leBIN(char *nome, int n, float mat[n][n]){
    FILE *f;
    int x[2];
    float v;

    f = fopen(nome, "rb");
    if(f==NULL){
      printf("Erro no acesso ao ficheiro\n");
      return;
    }
    while(fread(x, sizeof(int), 2, f) == 2){
        fread(&v, sizeof(float), 1, f);
        if(x[0]>=0 && x[0]<n && x[1]>=0 && x[1]<n)
            mat[x[0]][x[1]] = v;
    }
    fclose(f);
}
```

Exercício

- Alterar as funções que lidam com o array de clientes do banco, de modo a que a informação seja armazenada num ficheiro de texto *"clientes.txt"*

- Implementar as funções:

```
void guarda_dados_txt(cliente *b, int total);

cliente* recupera_dados_txt(int *total);
```

Operações Diretas em Ficheiros

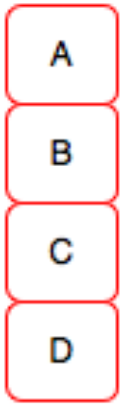
Operações diretas em ficheiros

- Utilizar ficheiro binário de clientes do banco como exemplo
  - Versão sem valor inteiro no início

1. Listar todos os clientes
2. Atualizar montante de um cliente
3. Adicionar um novo cliente

```
typedef struct dados cliente;
struct dados {
   char nome[100];
   char nconta[15];
   int montante;
};
```

banco.dat



Operação direta 1: Listar todos os clientes

```
void listarFicheiro(char *nome){
    FILE *f;
    cliente c;

    f = fopen(nome, "rb");
    if(f==NULL)
        return;

    while(fread(&c, sizeof(cliente), 1, f) == 1)
        escreve_info(c);

    fclose(f);
}
```

Operação direta 2: Atualizar o saldo de um cliente

```
int atualizaSaldoV1(char *nome, char *conta, int valor);
```

- Atualiza o saldo da conta *conta* armazenada no ficheiro binário *nome*
- Devolve sucesso (1) ou falhanço (0)

- Situação a considerar:
  - É uma operação que necessita que o ficheiro seja aberto para leitura e escrita

Operação direta 2A: Atualizar o saldo de um cliente

- Estratégia A
  - Utilizar um ficheiro temporário para onde é copiada a informação atualizada
  - No final
    - Apagar o ficheiro original.
    - Mudar o nome do ficheiro temporário.

```
int remove(char* nome);
int rename(char* antigo, char* novo);
```

Devolvem 0 se tudo correr bem

Operação direta 2A: Atualizar o saldo de um cliente

```
int atualizaSaldoV1(char *nome, char *conta, int valor){
    FILE *f, *g;
    cliente c;
    int flag = 0;

    f = fopen(nome, "rb");
    if(f == NULL) return 0;
    g = fopen("temp.dat", "wb");
    if(g == NULL){ fclose(f); return 0;}
    while(fread(&c, sizeof(cliente), 1, f) == 1){
        if(strcmp(c.nconta, conta) == 0){
            c.montante += valor;
            flag = 1;
        }
        fwrite(&c, sizeof(cliente), 1, g);
    }
    fclose(f); fclose(g);
    remove(nome); rename("temp.dat", nome);
    return flag;
}
```

Operação direta 2B: Atualizar o saldo de um cliente

isec
Politécnico de Coimbra

- Estratégia B
  - Abrir um ficheiro para leitura e escrita
    - Modos: r+, w+ ou a+
  - Trocar de leitura para escrita
    - Reposicionar o acesso
  - Trocar de escrita para leitura
    - Reposicionar o acesso
    - Esvaziar o buffer de output

Reposicionamento do Ponto de Acesso

isec Politécnico de Coimbra

A função seguinte muda o ponto de acesso ao ficheiro

Ficheiro

```
int fseek(FILE *f, long nBytes, int origem);
```

Devolve 0 se correr bem

Novo ponto de acesso passa a ser origem + nBytes

nBytes pode ser positivo ou negativo

Local a partir do qual é calculado o novo ponto de acesso:

- **SEEK_CUR:** posição atual
- **SEEK_END:** fim do ficheiro
- **SEEK_SET:** início do ficheiro

Determinar a posição do ficheiro

```
long ftell(FILE *f);
```

Devolve a posição atual (em bytes) no ficheiro

Operação direta 2B: Atualizar o saldo de um cliente

```
int atualizaSaldoV2(char *nome, char *conta, int valor){
    FILE *f;
    cliente c;

    f = fopen(nome, "rb+");
    if(f == NULL)
        return 0;

    while(fread(&c, sizeof(cliente), 1, f) == 1){
        if(strcmp(c.nconta, conta) == 0){
            c.montante += valor;
            fseek(f, -sizeof(cliente), SEEK_CUR);
            fwrite(&c, sizeof(cliente), 1, f);
            fclose(f);
            return 1;
        }
    }
    fclose(f);
    return 0;
}
```

Operação direta 3: Adicionar um novo cliente

```
int adicionaCliente(char *nomeFich, cliente novo);
```

- Restrição importante
  - O ficheiro está ordenado por número de conta e deve permanecer ordenado depois da adição do novo cliente

- Estratégias possíveis
  1. Usar um ficheiro temporário
  2. Abrir o ficheiro para leitura/escrita